Tropical Regression and Tropical Geometry Applied to Machine Learning

TAMRI Adnane Télécom SudParis, Institut Polytechnique de Paris

CONTENTS

1 Introduction 3
- 1.1 Tropical Operations . . . 3
- 1.2 Tropicalization and Dequantization . . . 3
- 1.3 Example of a Tropical Polynomial Curve . . . 3
- 1.4 Tropical Curves and Newton Polytopes . . . 4
- 1.5 A New Geometry for Learning . . . 4

2 Tropical Regression in 1D 4
- 2.1 Overview of the Method . . . 5
- 2.2 Convergence and Results . . . 6
- 2.3 Regression for Degree 1 Tropical Polynomial . . . 6
  - 2.31 Initial vs Final Iteration . . . 6
  - 2.32 Global Error and Threshold Evolution . . . 6
- 2.4 Regression for Degree 2 Tropical Polynomial . . . 7
  - 2.41 Initial vs Final Iteration . . . 7
  - 2.42 Global Error and Threshold Evolution . . . 7
- 2.5 Regression for Degree 4 Tropical Polynomial . . . 8
  - 2.51 Initial vs Final Iteration . . . 8
  - 2.52 Global Error and Threshold Evolution . . . 8
- 2.6 Conclusion of the 1D Regression Study . . . 8

3 Tropical Regression in Two Dimensions 9
- 3.1 2D Regression Strategy via Local Plane Propagation . . . 9
- 3.2 Results in 2D Tropical Regression . . . 10
  - 3.21 Case 1: Degree 1 Tropical Polynomial . . . 10
  - 3.22 Case 2: Degree 2 Tropical Polynomial . . . 11
  - 3.23 Case 3: Degree 5 Tropical Polynomial . . . 12
- 3.3 Conclusion of 2D Regression . . . 12

4 Principal Component Analysis and Tropical Regression on MNIST 13
- 4.1 Dimensionality Reduction via PCA and Low-Dimensional Tropical Modeling . . . 13

5 Tropical Classification and Empirical Evaluation 16
5.1 Model Design and Training Procedure 16
5.2 Experimental Setup and Training Procedure 17
5.3 Impact of Batch Size on Tropical Learning Dynamics 18
5.4 Model Performance and Class-wise Analysis 20
5.41 Training Time as a Function of Epochs 20
5.42 Confusion Matrix and Per-Class Accuracy 20
5.5 Comparison with Classical Machine Learning Models 22

6 Conclusion 22

References 23

1. Introduction

Tropical geometry is a recent field at the intersection of algebraic geometry, convex geometry, and discrete mathematics. It is based on a fundamental modification of classical arithmetic: instead of using the usual addition and multiplication, these operations are replaced by so-called *tropical operations*.

A. Tropical Operations

There are two main variants:

- **Max-plus**: addition is replaced by `max`, and multiplication by addition:

$$a \oplus b = \max(a, b), \qquad a \otimes b = a + b.$$

- **Min-plus** (dual): addition becomes `min`, while multiplication remains addition:

$$a \oplus b = \min(a, b), \qquad a \otimes b = a + b.$$

These structures are known as *tropical semirings*, since they do not include inverse elements.

B. Tropicalization and Dequantization

Tropicalization is a transformation process that consists in turning a classical algebraic function into its tropical version. It can be interpreted as a logarithmic limit, called *Maslov dequantization*:

$$\lim_{\theta \to 0} \theta \cdot \log\left(e^{a/\theta} + e^{b/\theta}\right) = \max(a, b).$$

This transformation produces angular geometric objects composed of linear pieces, while preserving key properties of their classical analogs.

C. Example of a Tropical Polynomial Curve

To demonstrate the effect of tropicalization, we compare a classical polynomial with its tropical counterpart in one dimension.

Consider the quadratic polynomial:

$$p(x) = x^2 + 2x + 2,$$

which describes a smooth parabolic curve.

Applying tropicalization in the max-plus semiring leads to the following tropical polynomial:

$$p_{\text{trop}}(x) = \max(2x, x + 2, 2),$$

which is a piecewise-linear function composed of three affine segments. The regions where the dominant term changes correspond to the tropical roots—the breakpoints of the curve.

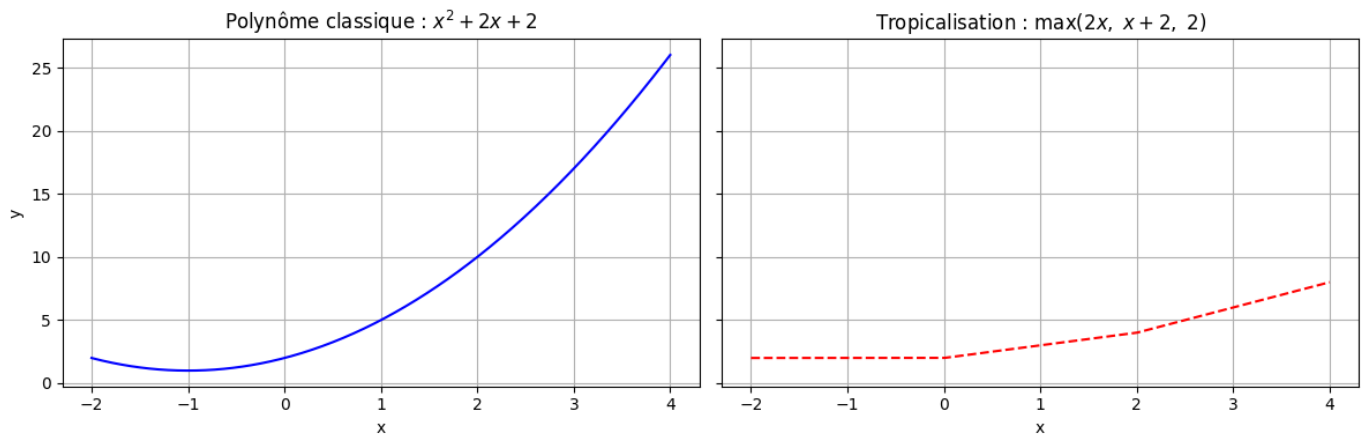


Fig. 1: Comparison of classical and tropical polynomial forms. **Left:** Traditional parabola $p(x) = x^2 + 2x + 2$. **Right:** Tropical version $p_{\text{trop}}(x) = \max(2x, x + 2, 2)$, consisting of linear segments joined at breakpoints.

D. Tropical Curves and Newton Polytopes

Tropical curves correspond to the loci of non-differentiability of a tropical function, i.e., the points where multiple monomials simultaneously attain the maximum:

$$V(p) = \left\{ x \in \mathbb{R}^d \;\middle|\; \text{at least two terms of } p(x) \text{ reach the maximum} \right\}.$$

A *Newton polytope* can also be associated with a tropical polynomial. It is defined as the convex hull of the slope vectors (i.e., exponents) of the monomials. These objects are fundamental for understanding the combinatorial structure of linear regions.

E. A New Geometry for Learning

This piecewise-linear formalism makes tropical geometry particularly well suited for analyzing many machine learning methods:

- Regression methods based on maximum of affine functions,
- Classification methods based on maximum of affine functions,
- Neural networks with ReLU activation functions,

In this paper, we focus primarily on the first and second application—**regression and classification using max-affine models**—where tropical geometry naturally applies. The third application, although relevant, is not addressed in this work.

2. Tropical Regression in 1D

In this section, we present a practical method for performing tropical regression in one dimension. The goal is to reconstruct a tropical polynomial from noisy observations of its values over a real interval.

This task is central to tropical learning, as it combines two key ideas: estimating the number and positions of the function's linear regions (i.e., its breakpoints), and fitting affine segments to each region in a way that approximates the data as closely as possible. Unlike classical polynomial regression, which assumes smoothness, tropical regression seeks to identify sharp transitions and model the signal using max-affine expressions.

We describe an iterative algorithm that alternates between estimating the positions of breakpoints and fitting linear pieces, and demonstrate its effectiveness on synthetic data of increasing complexity.

A. Overview of the Method

The regression begins with a **clustering step** to estimate the number of linear pieces composing the tropical function. If we detect k clusters, we infer that the polynomial has k linear segments and thus $k-1$ breakpoints (referred to as *thresholds*). The goal is to iteratively fit the best piecewise-linear function aligned with these segments.

To do so, we develop a custom algorithm that alternates between approximating the data with local linear segments and adjusting the threshold positions to minimize the total approximation error. Each segment is fitted using simple linear regression on selected anchor points, and the error is measured using a **mean squared error** (MSE) criterion. Thresholds are then updated in the direction of higher error to refine the partitioning.

This method is applied for tropical polynomials of increasing degree, starting with $k = 2$ and progressively extending to more complex functions. This allows us to assess how the approximation quality evolves with model complexity.

Algorithm 1 Tropical Piecewise Regression via Threshold Adjustment

Input: Data points (x_i, y_i), number of segments k, step size parameter $\alpha > 1$, maximum iterations T
Initialize: Place $k-1$ thresholds evenly over the input domain
for $t = 1$ to T **do**
    Select anchor points:
    for each segment s **do**
        if s is first or last segment **then**
            Use the median of the first or last 10% of data
        else
            Use a representative point from each of the two adjacent threshold zones
        end if
    end for
    Piecewise Linear Approximation:
    Connect the anchor points to define regression lines for each segment
    Compute errors (MSE):
    for each segment with predicted values y_{pred} and true values y **do**
        Compute **Mean Squared Error**:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - y_{\text{pred},i})^2$$

    end for
    Update thresholds:
    for each threshold t_j **do**
        Let $e_{\text{left}}, e_{\text{right}}$ be MSEs on adjacent segments
        Shift t_j in the direction of larger error:

$$t_j \leftarrow t_j + \alpha \cdot \text{sign}(e_{\text{right}} - e_{\text{left}})$$

        where $\alpha > 1$ controls the magnitude of the shift
    end for
end for
Output: Final piecewise-linear approximation and threshold locations

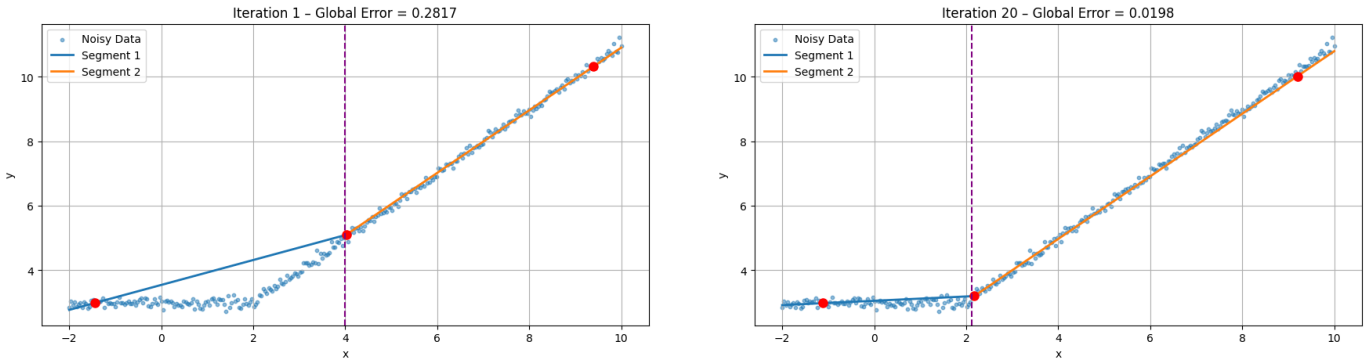
B. Convergence and Results

We evaluate the performance of our tropical regression method across three cases:

- A tropical polynomial of degree 1 (2 linear pieces),
- A tropical polynomial of degree 2 (3 linear pieces),
- A tropical polynomial of degree 4 (5 linear pieces).

For each scenario, we simulate noisy data and run the regression algorithm as previously described.

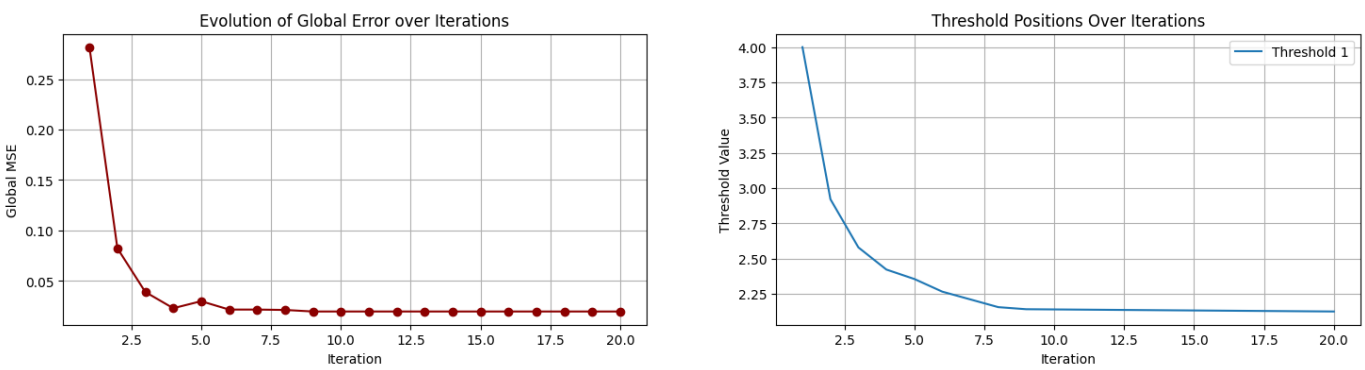
C. Regression for Degree 1 Tropical Polynomial



Initial regression Final regression

Fig. 2: Initial and final regression for degree 1 tropical polynomial

1) Initial vs Final Iteration: We observe that the algorithm progressively refines the fit of the tropical polynomial. The initial estimate is crude, but after a few iterations the threshold aligns visually with the expected breakpoint, and the linear segments closely approximate the noisy data.



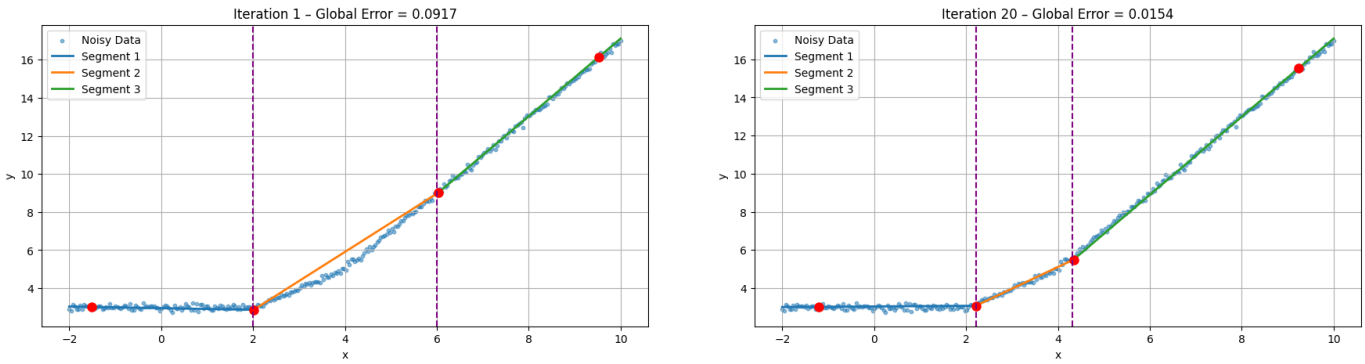
Global MSE over iterations Threshold position evolution

Fig. 3: Global error and threshold dynamics for degree 1 regression

2) Global Error and Threshold Evolution: The global mean squared error shows a rapid decrease, indicating that the threshold adjustment mechanism is working effectively. After around 8 iterations, the error stabilizes at a very low level.

The evolution of the threshold demonstrates smooth convergence toward the true position. Since the shift of each threshold is proportional to the local error, early iterations show larger movements, which gradually diminish as the segments become well-fitted.

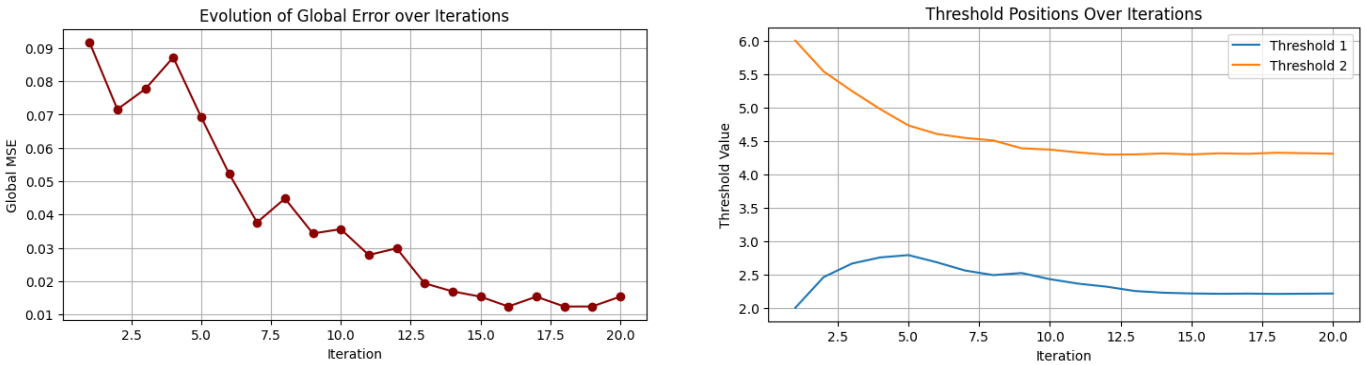
D. Regression for Degree 2 Tropical Polynomial



Initial regression Final regression

Fig. 4: Initial and final regression for degree 2 tropical polynomial

1) Initial vs Final Iteration: In this second-degree case, the regression starts from a basic initialization and successfully converges toward the true shape of the tropical polynomial. The final iteration shows three well-aligned linear pieces, closely following the underlying data and capturing the correct transitions between segments.



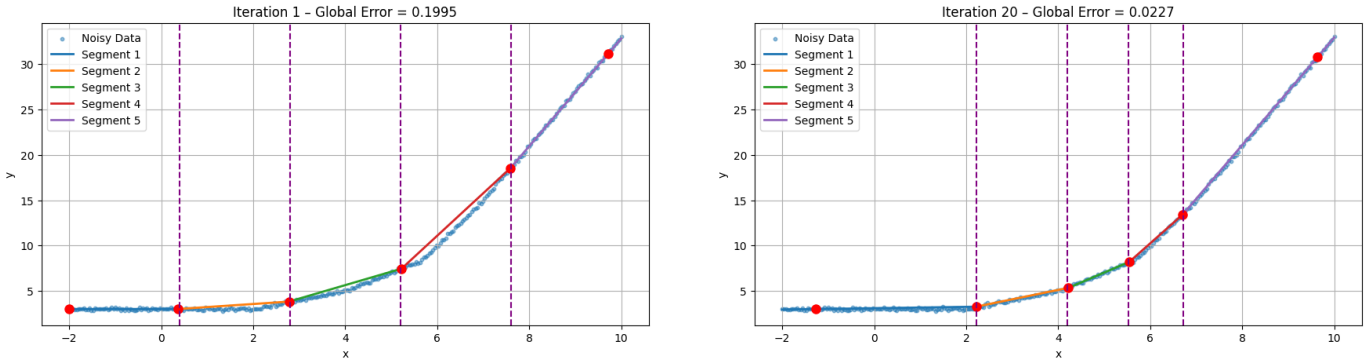
Global error convergence — degree 2 Threshold evolution — degree 2

Fig. 5: Global error and threshold evolution for degree 2 regression

2) Global Error and Threshold Evolution: The global error curve does not decrease monotonically. This is expected: when optimizing the fit on one side of a threshold, the error on the adjacent region may increase temporarily. Nevertheless, the overall trend is decreasing, and the system stabilizes after a few iterations.

The evolution of the thresholds shows an interesting dynamic: the first threshold quickly stabilizes, while the second one increases initially and then moves back, settling into its final optimal position. This adjustment process reflects the competition between the neighboring segments in minimizing their respective errors.

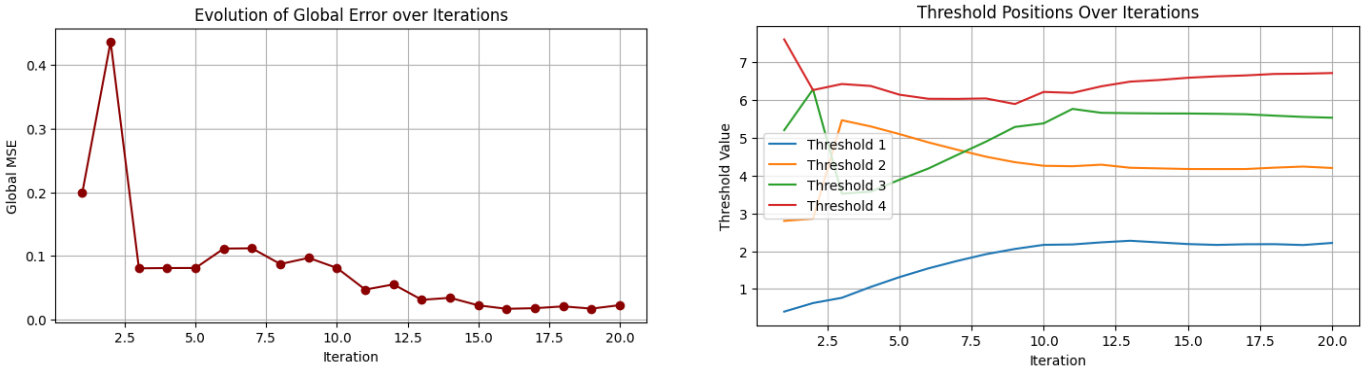
E. Regression for Degree 4 Tropical Polynomial



Initial regression Final regression

Fig. 6: Initial and final regression for degree 4 tropical polynomial

1) Initial vs Final Iteration: With five linear pieces, the degree 4 regression illustrates the scalability of our method. Visually, the breakpoints align well with the transition points of the tropical polynomial. Despite the complexity, the algorithm manages to produce a smooth and accurate reconstruction of the function.



Global error convergence — degree 4 Threshold evolution — degree 4

Fig. 7: Convergence diagnostics for degree 4 regression. **Left:** Global mean squared error per iteration. **Right:** Evolution of threshold positions.

2) Global Error and Threshold Evolution: The global error exhibits a sharp increase at the second iteration, likely caused by a poor early threshold adjustment. However, after this perturbation, the algorithm recovers and gradually improves the fit, reaching stability after roughly 16 iterations.

The threshold trajectories show multiple inversions—each breakpoint changes direction more than once before converging. This back-and-forth motion reflects the complex trade-off between adjacent regions, with convergence finally achieved around iteration 15.

F. Conclusion of the 1D Regression Study

Our experimental study demonstrates the effectiveness of tropical regression for reconstructing piecewise linear tropical polynomials from noisy data. Across all tested degrees—ranging from simple (degree 1) to more complex functions (degree 4)—the algorithm successfully identifies the correct number of linear regions and converges toward an accurate approximation.

Key observations include:

- The thresholds tend to stabilize near the true breakpoints, even in the presence of noise.
- The global error consistently decreases, confirming the convergence of the regression.
- The adjustment strategy, which moves breakpoints in proportion to local errors, enables the model to refine the fit adaptively across iterations.

These results validate the suitability of this approach for tropical regression in one dimension. In the next section, we extend this framework to tropical surfaces in two dimensions.

3. Tropical Regression in Two Dimensions

In this section, we extend the tropical regression approach to functions of two variables. Tropical polynomials in dimension two correspond to the maximum of several affine planes:

$$f(x, y) = \max(a_1 x + b_1 y + c_1,\ \ldots,\ a_k x + b_k y + c_k).$$

Each region of the domain is governed by a different linear function, and the boundaries between regions are tropical curves—piecewise linear ridges where two or more planes intersect.

Our objective is to reconstruct such a tropical surface from noisy data points $\{(x_i, y_i, z_i)\}$ sampled from a hidden max-affine function.

To better understand the geometric structure of such surfaces, it is useful to visualize a typical tropical surface. In the tropical setting, each affine term $a_i x + b_i y + c_i$ defines a planar region, and the tropical polynomial is obtained by taking the maximum of these planes. The resulting surface is composed of flat polygonal facets joined along sharp ridges where two or more affine terms are simultaneously maximal. These ridges form the so-called *tropical curves*, which act as the boundaries between linear regions.

An example of such a surface is illustrated below:

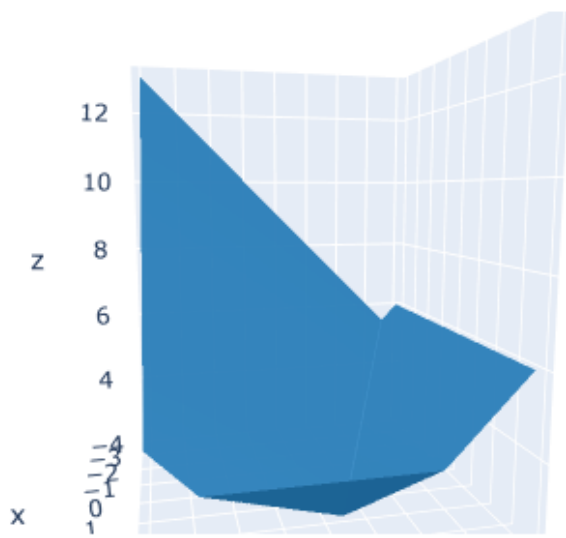


Fig. 8: Illustration of a tropical surface in $\mathbb{R}^2$. The function is composed of several planar pieces, and the sharp ridges between them form the tropical curves.

A. 2D Regression Strategy via Local Plane Propagation

In contrast to the 1D case, tropical regression in two dimensions is handled through an iterative geometric process. The algorithm is designed to identify and reconstruct the piecewise-linear components of a tropical surface based on local plane fitting and neighborhood propagation. The overall procedure follows these steps:

Algorithm 2 Tropical Surface Fitting in 2D via Local Max-Affine Planes

Input: Noisy dataset $\{(x_i, y_i, z_i)\}_{i=1}^{n}$ sampled from a tropical surface.
Parameters: Tolerance ε, neighborhood radius r.
Initialize all points as `unassigned`.
while there are unassigned points **do**
    Select a random unassigned point P.
    Identify all points within radius r of P.
    Fit an affine plane to this local neighborhood using linear regression.
    Compute vertical distance of all points to this plane.
    Mark as `assigned` all points whose distance is $\leq \varepsilon$.
    Store the fitted plane as part of the final tropical surface.
end while
Output: A collection of affine planes approximating the tropical surface.

This method is particularly well-suited to tropical geometry, as it leverages the fact that tropical polynomials are composed of convex regions formed by maxima of affine functions. The algorithm is robust to noise and provides an interpretable segmentation of the tropical surface into its linear parts.

B. Results in 2D Tropical Regression

We now present the outcome of applying the local plane propagation algorithm to several tropical surfaces in dimension 2. Each surface is generated from a tropical polynomial with different numbers of affine components and corrupted with Gaussian noise.

1) Case 1: Degree 1 Tropical Polynomial: In the case of a tropical polynomial of degree 1, the model successfully identifies two distinct planar regions corresponding to two max-affine components. One of these is nearly flat, while the other has a noticeable slope. Together, they accurately reconstruct the underlying piecewise-linear structure, even in the presence of noise. The visual segmentation obtained by the regression is shown below:

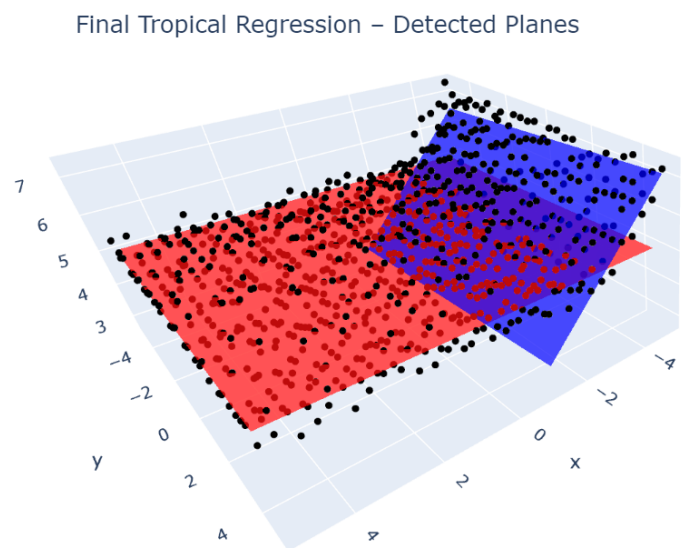


Fig. 9: Detected planes for a degree 1 tropical polynomial. The surface is composed of two affine regions clearly separated by a tropical ridge.

The equations of the two detected planes, estimated through local linear fitting, are presented in the following table:

TABLE I: Equations of the detected planes (Degree 1)

Plane	Equation
Plane 1	$z = 0.004x + 0.023y + 5.115$
Plane 2	$z = -1.001x + 0.057y + 1.806$

To assess the quality of the fit, we report the mean squared error (MSE) for each region, both with respect to the noisy data and the true underlying function. The results are summarized below:

TABLE II: Error analysis for degree 1 regression

Plane	MSE (Noisy)	MSE (True)	# Points
Plane 1	0.1619	0.1346	850
Plane 2	0.3765	0.3805	303

We observe that the second plane (associated with a smaller region) has a significantly higher error. This is likely due to the lower number of points available for fitting, which limits the accuracy of the local regression. With denser sampling or adaptive point selection, this approximation could likely be improved.

2) Case 2: Degree 2 Tropical Polynomial: The result for the degree 2 tropical polynomial is also satisfactory: the model correctly detects 3 planar regions, which aligns with the true number of affine components. All data points are clearly associated with one of the fitted planes, showing a clean segmentation of the surface. This behavior is consistent with the observations made in the degree 1 case.

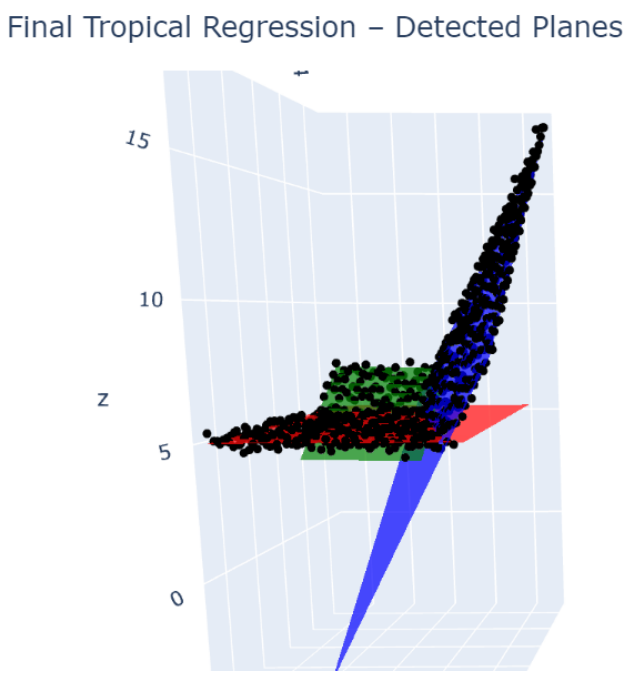


Fig. 10: Detected planes — degree 2 tropical polynomial. A total of 4 planar regions were identified.

TABLE III: Detected Plane Equations (Degree 2)

Plane	Equation
Plane 1	$z = 0.020x + 1.970y + 1.969$
Plane 2	$z = 1.997x + 0.040y + 1.991$
Plane 3	$z = -0.010x + 1.000y + 1.989$

However, in this case, we observe a slightly larger fitting error, particularly a noticeable discrepancy between the true plane and the detected one for some regions. This suggests that while the global structure is well captured, local noise or geometric overlaps may affect the precision of certain segments.

TABLE IV: Error Analysis — Degree 2

Plane	MSE (Noisy)	MSE (True)	# Points
Plane 1	0.1666	0.1530	335
Plane 2	0.1508	0.1347	348
Plane 3	0.2261	0.2584	117

3) Case 3: Degree 5 Tropical Polynomial: In the degree 5 case, the algorithm detects a larger number of planes, as expected. However, we observe that some of these planes appear to overlap, especially near the transition regions. This is mainly due to the presence of noise, which makes it more difficult to separate high-degree tropical surfaces into clearly segmented linear components. Despite this, the overall structure remains consistent with the expected piecewise-affine pattern.

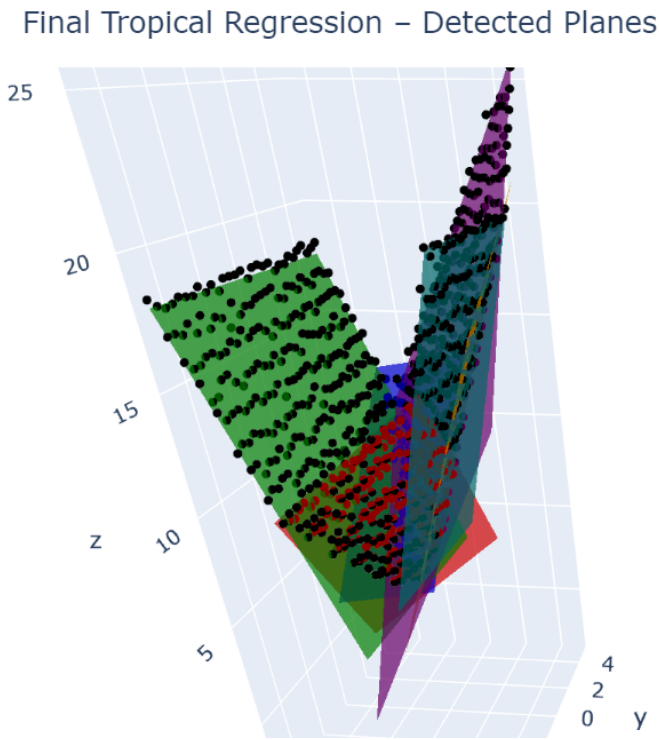


Fig. 11: Detected planes — degree 5 tropical polynomial. The method captured the structure despite overlapping and noise.

TABLE V: Detected Plane Equations (Degree 5)

Plane	Equation
Plane 1	$z = 0.833x - 1.911y + 2.340$
Plane 2	$z = 0.176x + 1.333y + 6.242$
Plane 3	$z = -3.022x - 0.035y + 2.765$
Plane 4	$z = 2.960x + 0.773y + 3.856$
Plane 5	$z = 2.010x + 1.969y + 6.100$
Plane 6	$z = 4.161x - 0.026y - 0.733$

C. Conclusion of 2D Regression

The local plane propagation method proves effective in reconstructing tropical surfaces from noisy data. For low to moderate degrees, the number of detected planes closely matches the true number of monomials, and the segmented regions exhibit coherent geometric structure.

TABLE VI: Error Analysis — Degree 5

Plane	MSE (Noisy)	MSE (True)	# Points
Plane 1	0.2586	0.2509	174
Plane 2	0.6549	0.6572	138
Plane 3	0.1147	0.0894	351
Plane 4	0.6139	0.6532	150
Plane 5	0.1226	0.1009	348
Plane 6	0.2119	0.2108	160

However, when the degree increases or when the noise level is too high, the model becomes more sensitive. In such cases, it may detect an excessive number of small planar regions, assigning nearly one plane per point. This results in longer runtimes and overfitting. These limitations highlight the need for additional regularization or post-processing techniques to merge redundant planes and better handle noise.

Overall, the method remains a powerful geometric tool for extracting max-affine structure in tropical settings.

4. Principal Component Analysis and Tropical Regression on MNIST

To explore the potential of tropical geometry for data analysis and classification, we focus on a well-known benchmark: the **MNIST** dataset. This dataset consists of 70,000 grayscale images of handwritten digits (0 to 9), each of size 28×28, resulting in 784-dimensional vectors when flattened. MNIST remains a standard testbed for evaluating machine learning models due to its simplicity, diversity, and relatively high dimensionality.

Our objective is to perform a form of **tropical clustering** or segmentation on this dataset. Specifically, we aim to identify regions in the data space where a tropical (max-affine) model could locally approximate the digit images. This raises several challenges:

- The dimensionality of the data ($d = 784$) is extremely high, while tropical methods tend to rely on geometric intuition that becomes difficult to visualize or apply directly in such spaces.
- Classical clustering methods (e.g., k-means) may not capture the structure aligned with tropical piecewise-linear regions.
- The volume of the data also raises computational concerns for any approach that requires fitting multiple tropical models.

We explore two strategies to address these issues:

1) **Dimensionality reduction via PCA**: We apply Principal Component Analysis to project the MNIST images onto a lower-dimensional space (e.g., $\mathbb{R}^2$ or $\mathbb{R}^3$), where we can reuse the tropical regression and segmentation techniques developed for low-dimensional cases.
2) **Direct high-dimensional tropical modeling**: We develop and evaluate tropical classification models that operate directly in the original 784-dimensional space, using piecewise max-affine functions and stochastic optimization methods. This allows us to bypass dimensionality reduction and study the behavior of tropical methods in their native high-dimensional setting.

Both strategies are implemented and compared. The first serves as an intuitive and visual introduction to tropical clustering, while the second demonstrates the viability of tropical classifiers for full-scale image classification tasks such as MNIST.

A. *Dimensionality Reduction via PCA and Low-Dimensional Tropical Modeling*

To make tropical modeling tractable on high-dimensional data such as MNIST, we first explore dimensionality reduction using Principal Component Analysis (PCA). The idea is to project the original 784-dimensional input space onto a lower-dimensional subspace—specifically, $\mathbb{R}^2$ or $\mathbb{R}^3$—in which geometric intuition and tropical regression models become more feasible.

However, the projection alone does not guarantee that the class structure—that is, the natural groupings of digits—remains preserved in the reduced space. To assess the relevance of this dimensionality reduction and to simplify the analysis in a first step, we choose to restrict our study to a subset of the data. More precisely, we focus on only three digit classes, selecting the trio that exhibits the best geometric separation after projection.

This strategy serves several purposes. It simplifies both visual and algorithmic analysis by reducing the dimensionality to $\mathbb{R}^3$, which is manageable both computationally and geometrically. It also helps us identify a favorable configuration for applying tropical models, and acts as a stepping stone toward the more complex task of modeling the full set of ten digit classes.

To select the optimal trio of digit classes for low-dimensional modeling, we systematically evaluate all $\binom{10}{3} = 120$ possible combinations. For each trio, we randomly sample a balanced subset of 50 images per class and apply PCA to reduce the data to three dimensions. We then compute the centroid of each class in the projected space and define a quantitative score that reflects the clarity of class separation.

This score, called the *centroid separation score*, is computed as the ratio between inter-class distance and intra-class compactness. More precisely, we first calculate the Euclidean distance between each pair of centroids, and take the average of these three distances:

$$D_{\text{inter}} = \frac{1}{3}\left(\|\mu_1 - \mu_2\| + \|\mu_1 - \mu_3\| + \|\mu_2 - \mu_3\|\right).$$

We then measure the intra-class dispersion by computing, for each class, the average Euclidean distance between each point and its class centroid, and then averaging over the three classes:

$$D_{\text{intra}} = \frac{1}{3}\sum_{i=1}^{3}\frac{1}{|C_i|}\sum_{x\in C_i}\|x - \mu_i\|.$$

The final score is defined as:

$$\text{Score} = \frac{D_{\text{inter}}}{D_{\text{intra}}}.$$

A high score indicates that the centroids of the three classes are well-separated in the reduced space, while each class remains tightly clustered around its center. This allows us to identify the most distinguishable digit combinations, which are ideal candidates for testing tropical segmentation methods. The top 10 trios with highest scores are listed below:

Digit Trio	**Centroid Separation Score**
(0, 1, 4)	2.81
(0, 1, 6)	2.62
(0, 1, 9)	2.62
(0, 1, 7)	2.61
(1, 6, 7)	2.56
(1, 3, 6)	2.54
(0, 2, 7)	2.53
(3, 6, 7)	2.52
(0, 1, 3)	2.50
(0, 1, 2)	2.46

Based on this ranking, we select the trio $(0, 1, 4)$ for further analysis. The PCA projection reveals a clear spatial separation between the three classes, making this subset suitable for tropical piecewise modeling.

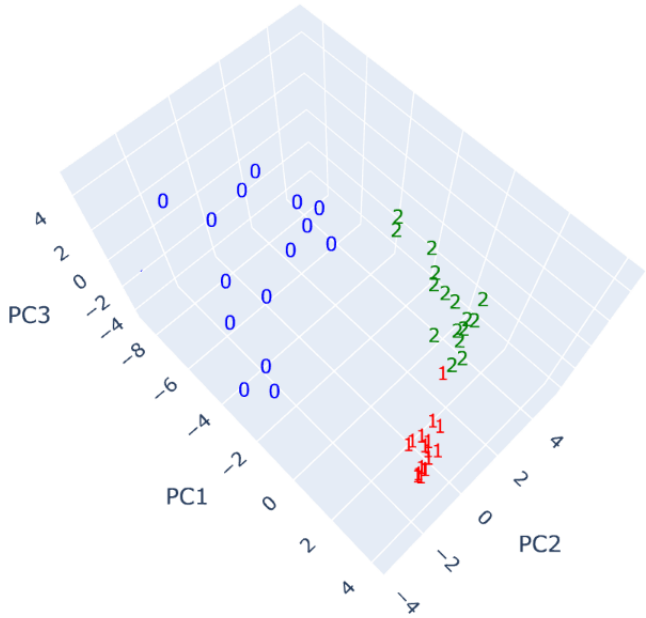


Fig. 12: PCA projection onto $\mathbb{R}^3$ of digit classes 0, 1, and 4. Each color corresponds to one class. The spatial separation observed here motivates the use of tropical modeling to approximate each class region with max-affine functions.

We then attempt to fit *tropical* (max–affine) regression planes to the 3-D point cloud of the optimal trio $(0, 1, 4)$. Starting from a random *seed* point, we build a local affine approximation (plane) and iteratively enlarge the region by adding nearby points whose vertical distance to the plane is below a tolerance. Although this heuristic offers an intuitive geometric segmentation, it is highly sensitive to the initial seed and to randomness; the resulting partitions can be unstable and only loosely follow the true class boundaries.

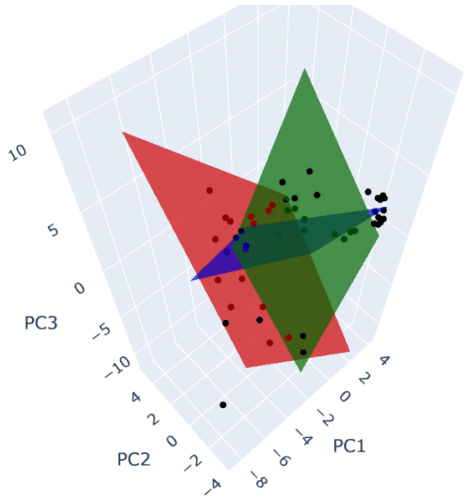


Fig. 13: Piecewise planar segmentation of the $(0, 1, 4)$ digits after PCA projection into $\mathbb{R}^3$, using tropical max-affine regression. The fitted planes approximate local regions but do not align well with class boundaries.

These limitations become even more pronounced when attempting to apply the same method to all ten digit classes simultaneously. After PCA projection into $\mathbb{R}^3$, the class clusters heavily overlap, making it nearly impossible to segment the space meaningfully using regression-based planes.

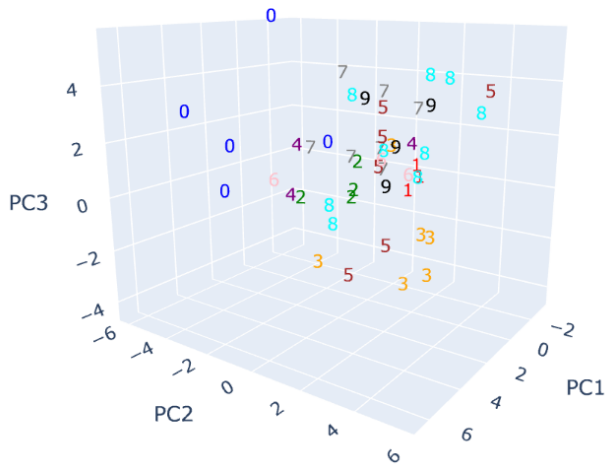


Fig. 14: PCA projection of all ten digit classes into $\mathbb{R}^3$. Each color corresponds to a digit from 0 to 9. The significant overlap between class clusters highlights the difficulty of applying tropical regression for full MNIST classification.

Our initial approach focused on applying tropical regression after PCA on MNIST data. However, even when restricting ourselves to a carefully selected subset of digits—chosen to optimize our regression score—we observed poor performance and unstable fits. The regression model struggled to capture meaningful structure in the projected data. Extending this approach to all digits using the same regression model led to inconclusive and unusable results. These limitations highlight a fundamental mismatch between regression and the categorical nature of the MNIST task. Consequently, we are compelled to abandon regression in favor of a tropical *classification* model. In the next section, we develop and implement this classifier, where each digit class is represented by its own max-affine scoring function.

5. Tropical Classification and Empirical Evaluation

Building upon the limitations encountered with tropical regression on PCA-projected MNIST data, we now turn to a more suitable framework: tropical **classification**. While regression struggled to capture the discrete structure of digit classes, classification offers a more natural formulation for such tasks.

In tropical classification, the goal is to assign each input to one of several predefined classes by computing a set of max-affine scoring functions—one per class—and selecting the class with the highest score. This approach aligns well with the piecewise-linear nature of tropical geometry and proves more robust when dealing with complex or overlapping class boundaries.

A. Model Design and Training Procedure

The tropical classifier used in our experiments is a **Tropical Max-Affine Classifier**. Each class $k \in \{0, \ldots, 9\}$ is associated with p affine components, and the class score is defined as:

$$\text{score}_k(x) = \max_{j=1,\ldots,p} \left(\langle a_k^{(j)}, x \rangle + b_k^{(j)} \right)$$

The predicted class is the one with the highest score.

To train this model, we define a tropical version of the classical **hinge loss**, widely used in Support Vector Machines (SVMs). The standard hinge loss encourages correct classification by enforcing a margin of at least 1 between the score of the true class and that of the most competitive incorrect class. Our **tropical hinge loss** adapts this principle to the tropical framework by replacing linear classifiers with tropical max-affine score functions.

Given the true class k and the predicted score vector $\mathbf{s} = (s_1, \ldots, s_n)$, the tropical hinge loss is defined as:

$$\mathcal{L}_{\text{trop}} = \max\left(0, 1 + \max_{j \neq k}(s_j) - s_k\right)$$

This loss behaves as follows:

- If the correct class k already has the highest score with a sufficient margin (greater than 1), then the loss is zero.
- If another class $j \neq k$ has a score close to or higher than the correct class, the loss becomes positive, penalizing the model accordingly.

Thus, the tropical hinge loss has two desirable effects:

- It **reduces intra-class variance** by encouraging consistent, confident scoring of correct labels.
- It **increases inter-class separation** by penalizing cases where incorrect classes are scored too closely to the correct one.

This formulation preserves the geometric nature of tropical models while aligning with classical classification objectives, making it well-suited for max-affine tropical classifiers.

B. Experimental Setup and Training Procedure

To evaluate the tropical classifier, we implement a lightweight training pipeline based on **stochastic heuristic optimization**. At each mini-batch iteration, random perturbations are applied to the model parameters (A, b) and accepted only if they decrease the tropical hinge loss, ensuring monotonic improvement without requiring gradients.

The Python implementation includes:

1) the `TropicalMaxAffineClassifier` class,
2) the tropical hinge loss function,
3) an epoch-wise training loop with rejection sampling,
4) multiple training repetitions across configurations,
5) and visualization of training curves for different batch sizes.

We focus on how the **batch size** affects convergence and performance, using values $\{8, 16, 32, 64, 128\}$. For each, we track:

- the number of epochs until convergence (early stopping),
- total training time,
- and update efficiency: the proportion of accepted parameter changes.

C. Impact of Batch Size on Tropical Learning Dynamics

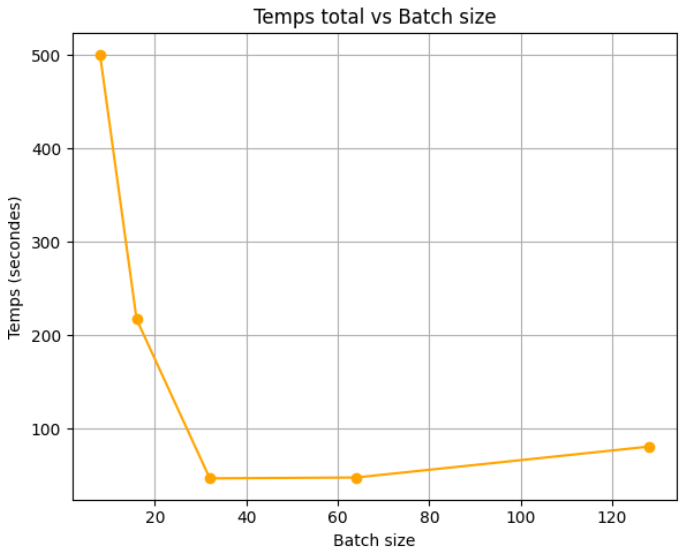


Fig. 15: Total training time vs batch size. Batch 32 minimizes execution time, balancing update frequency and stability.

Figure 15 shows that training time drops sharply up to batch size 32, then slightly increases. Small batches trigger too many updates; large batches underutilize updates, slowing down progress. Batch 32 offers the best time-efficiency trade-off.

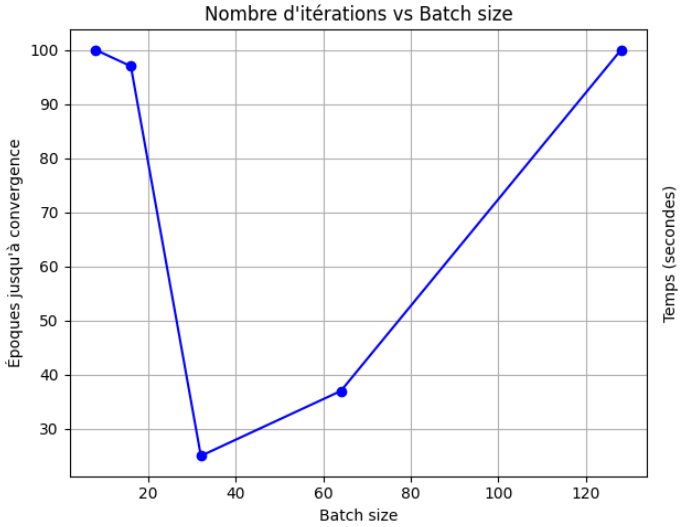


Fig. 16: Epochs until convergence vs batch size. A minimum is reached at batch 32.

The number of epochs in Figure 16 follows a U-shape, confirming batch 32 as optimal. Small batches are noisy; large ones converge slowly. Batch 32 combines frequent updates with stable progress.

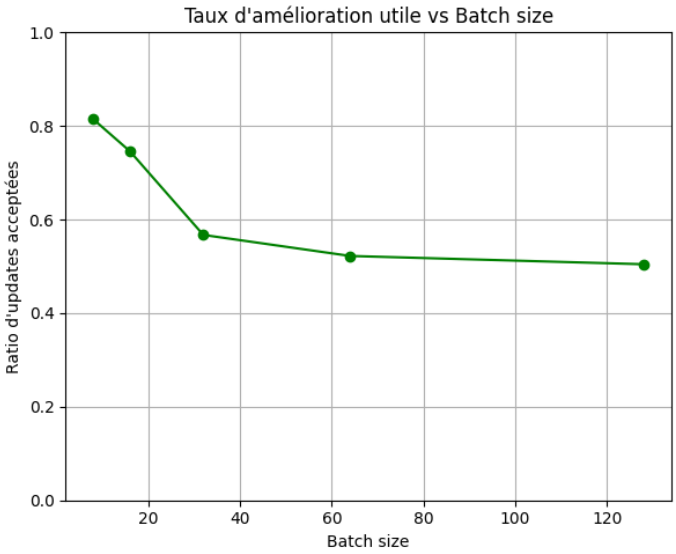


Fig. 17: Proportion of accepted parameter updates vs batch size.

Figure 17 shows that update efficiency decreases with batch size. Larger batches make the loss landscape smoother, reducing the chance that perturbations improve it. Still, all batch sizes remain above 0.5, and batch 32 again strikes the best balance.

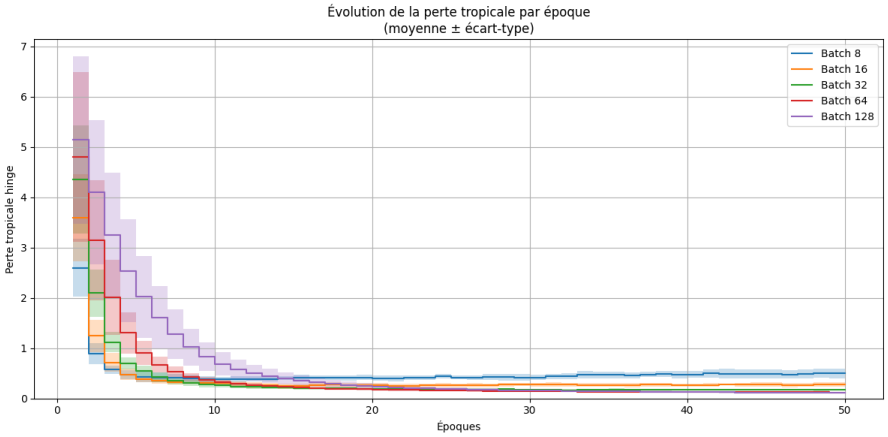


Fig. 18: Tropical hinge loss across epochs for different batch sizes. Shaded areas indicate standard deviation across runs.

In Figure 18, several dynamics emerge:

- **Batch 8** (blue) drops quickly but plateaus early with high final loss.
- **Batch 16** (orange) improves over batch 8 but still underperforms in the long run.
- **Batches 64 and 128** (red, violet) are slow to start, but reach low final loss values.
- **Batch 32** (green) achieves fast convergence and strong final performance, reaching low loss in only 15 epochs.

Conclusion. All indicators—execution time, convergence speed, and update efficiency—highlight **batch size 32** as the most effective configuration. It balances the exploratory power of small batches with the stability of larger ones.

This consistent outcome suggests a possible structural link between tropical optimization dynamics and batch granularity. Batch 32 may align with a natural partitioning scale in the tropical model's piecewise affine structure. Regardless, tuning batch size proves essential for efficient tropical learning pipelines.

D. Model Performance and Class-wise Analysis

After identifying the optimal batch size configuration, we evaluate the classification performance of the tropical max-affine model on the MNIST test set. Our analysis includes both global metrics and class-specific behaviors.

1) Training Time as a Function of Epochs: To ensure that our training routine scales well with longer training durations, we measure the total runtime of the algorithm as a function of the number of epochs. As shown in Figure 19, the relationship is approximately linear, confirming the expected computational complexity of the optimization procedure.

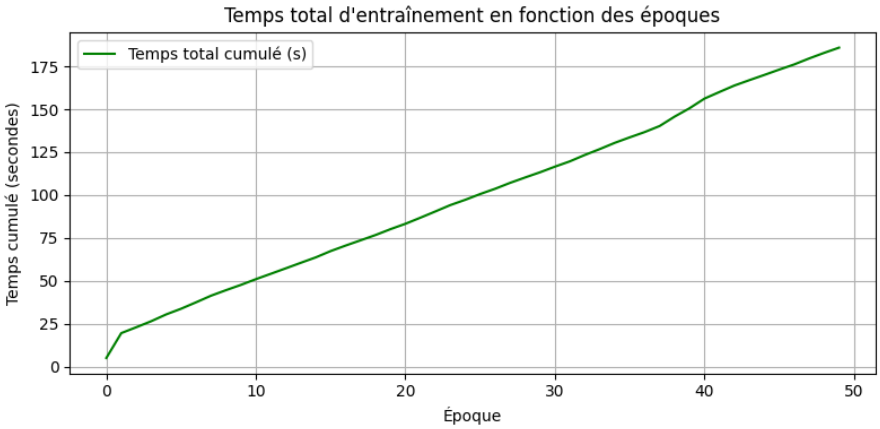


Fig. 19: Total execution time vs number of training epochs. The linear profile confirms the scalability of the optimization routine.

2) Confusion Matrix and Per-Class Accuracy: To better understand how the tropical classifier performs across different digit classes, we examine both the confusion matrix and the per-class accuracy. These tools allow us to analyze the strengths and weaknesses of the model beyond global accuracy.

The confusion matrix shown in Figure 20 reveals how often each digit is confused with others. Digit **1** is the most accurately predicted, with minimal confusion. In contrast, digit **5** suffers the most from misclassification, especially being confused with **3** and **8**. Digit **9** also performs poorly, often being mistaken for **4**, **7**, or **8**.

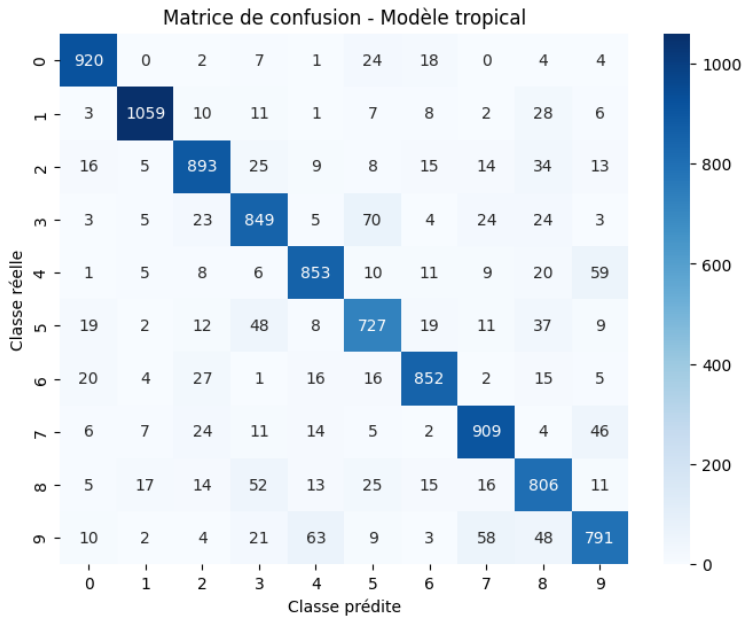


Fig. 20: Confusion matrix for the tropical classifier on MNIST. Rows represent true labels; columns represent predicted labels.

To complement this, Figure 21 illustrates the per-class accuracy. Digit **1** achieves the best score at **96%**, while digit **5** is the worst with only **79%**. Digits **0**, **1**, **2**, **4**, **6**, and **7** show accuracy above **90%**, while digits **3**, **5**, **8**, and **9** fall into the lower range (between **79%** and **85%**).

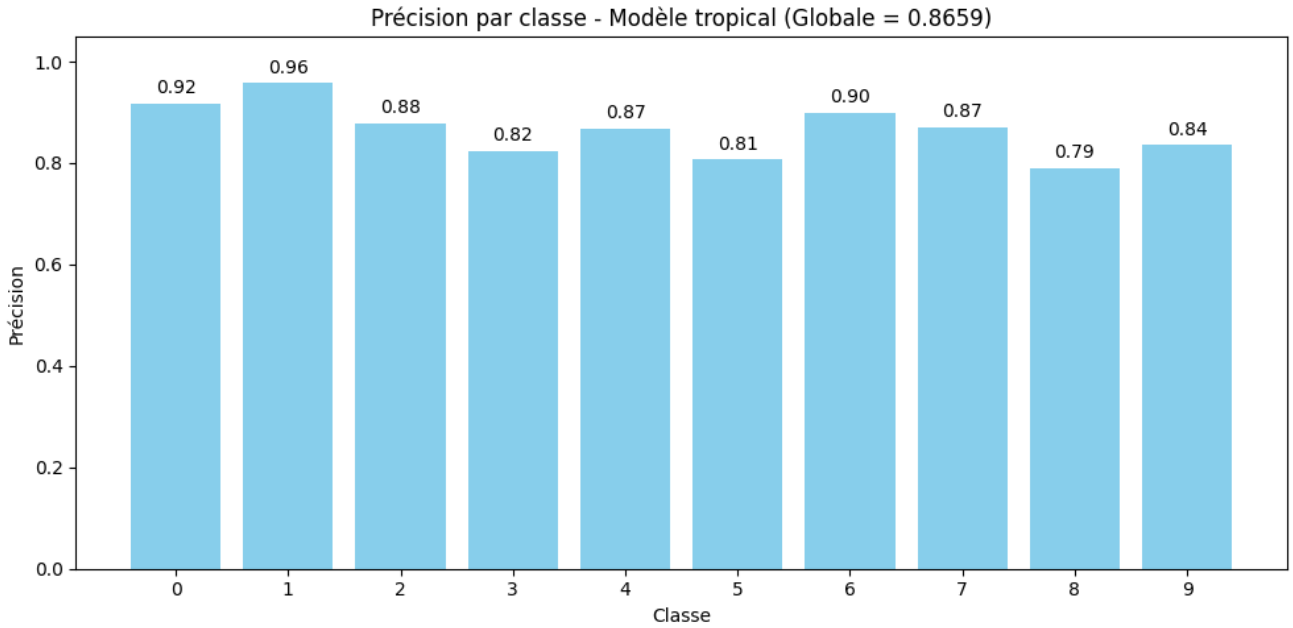


Fig. 21: Per-class accuracy for the tropical classifier. Each bar corresponds to one digit (0 through 9).

Overall, these results confirm a good performance of the tropical model with a global test accuracy of **86%**. Despite some heterogeneity across classes, the classifier remains competitive and reveals meaningful structure in high-dimensional data using only tropical max-affine components.

E. Comparison with Classical Machine Learning Models

To contextualize the performance of our tropical classifier, we compare it with several well-known classification models from scikit-learn: Random Forest, K-means, K-medoids, and Gaussian Mixture Models (GMM). All models are trained and evaluated on the same MNIST subset for fair comparison.

This comparison serves a dual purpose:

- to assess whether tropical models provide a viable alternative to classical ensemble or clustering-based approaches;
- to understand how tropical methods behave under similar computational constraints and accuracy benchmarks.

The results are summarized below:

TABLE VII: Accuracy comparison across models on MNIST (same data subset)

Model	**Accuracy (%)**
Random Forest (scikit-learn)	97
Gaussian Mixture Model (GMM)	96
K-means Clustering	95
K-medoids Clustering	94
Tropical Max-Affine Classifier	**87**

Although the tropical classifier does not reach the top accuracy scores of models like Random Forest or GMM, its performance remains competitive when compared to classical clustering-based methods such as K-means or K-medoids. More importantly, it offers a fundamentally different modeling perspective, grounded in tropical geometry.

By relying on explicit max-affine functions and piecewise-linear decision boundaries, the tropical classifier introduces a novel and interpretable geometric framework for classification. While the current results are encouraging, there is still significant room for improvement—both in terms of accuracy and optimization techniques.

6. Conclusion

This work provides a comprehensive exploration of tropical geometry applied to regression, classification, and surface reconstruction. We began with foundational definitions and illustrated the transformation from classical to tropical polynomials, highlighting the piecewise-linear nature that makes tropical models inherently interpretable and geometrically structured.

In one dimension, we proposed an iterative threshold adjustment algorithm capable of fitting noisy tropical polynomials of increasing degree. The results demonstrated accurate recovery of breakpoints and low final approximation errors. In two dimensions, we introduced a local plane propagation method to reconstruct tropical surfaces from point clouds. This technique successfully segmented the domain into affine patches aligned with the underlying tropical structure, even in the presence of noise.

Moving to high-dimensional data, we applied tropical methods to the MNIST digit classification problem. After reducing the dimensionality using PCA, we first attempted tropical regression in $\mathbb{R}^3$, which revealed structural limitations due to class overlap. This motivated a transition to full tropical classification using max-affine score functions trained with tropical hinge loss and stochastic optimization.

Our experiments revealed that a batch size of 32 offers the best trade-off between convergence speed, training time, and update efficiency. The tropical classifier achieved a global accuracy of 86%, with detailed insights provided by the confusion matrix and per-class performance analysis. Though slightly outperformed by classical models like Random Forest, the tropical approach offers unique geometric insights and compact representation.

Overall, tropical geometry proves to be a promising framework for interpretable and piecewise-linear modeling. Future directions include extending tropical learning to unsupervised tasks, improving optimization heuristics, and applying these methods to other domains such as signal decoding or structured decision-making.

REFERENCES

[1] L. Zhang, G. Naitzat, and L.-H. Lim, "Tropical Geometry of Deep Neural Networks," in *Proceedings of the 35th International Conference on Machine Learning*, 2018.

[2] P. Maragos, V. Charisopoulos, and E. Theodosis, "Tropical Geometry and Machine Learning," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 728–755, 2021.